SITS 2026

奇点智能技术大会

Singularity Intelligence Technology Summit

主办方 / Organizers: 奇点智能研究院 Singularity Intelligence Research Institute CSDN

合作伙伴 / Partners: CECW 中能拾贝 OMNI-AI 乐奇 Rokid 异步社区 www.epubit.com 清華大學出版社 TSINGHUA UNIVERSITY PRESS

2026技术大会活动预告

奇点智能研究院 Singularity Intelligence Research Institute CSDN

SITS 2026

奇点智能技术大会

Singularity Intelligence Technology Summit

4月17-18日·上海 | 9月11-12日·北京

PM – Summit 2026

全球产品经理大会

Product Management Summit

7月17-18日·北京

CPP – Summit 2026

全球系统软件技术大会

C++ and System Software Summit

11月20-21日·上海

GOSIM®

GOSIM 2026

5月4-6日·巴黎 | 10月16-18日·深圳

1024程序员日

智汇深圳"人工智能+"创新大赛

AI构建世界 智能引领未来

10月24-25日·多城联动

媒体服务——多形态媒体矩阵

奇点智能研究院 Singularity Intelligence Research Institute　CSDN

为客户提供定制化内容服务，包括但不限于视频、文章、评测、专题报告、白皮书等多品类内容

访谈 → 新闻稿件 → 产品解读 → 产品评测 → 客户案例 → 活动报道 → 专题

直播栏目

《AI进化论》

《涛滔不绝》

《万有引力》

文章栏目

《AGI技术50人》

《近匠》

《开源英雄》

权威出版

《新程序员》

聚集全球技术大师、行业技术先锋，从前沿技术到行业先锋，为您带来全方位的深刻洞见和专业解读。

《专题报告》

《AI原生软件研发成熟度模型AISMM》
《2025全球开源发展报告》
云原生、AI赋能软件研发、数据库、操作系统等领域报告

技术咨询——企业信赖的策略伙伴

奇点智能研究院 Singularity Intelligence Research Institute CSDN

依托雄厚的专家团队和丰富的行业经验，通过12大技术咨询板块，提供涵盖人工智能、架构设计、产品管理等全栈技术咨询解决方案，帮助企业在数字化、智能化、万物互联的变革中打造关键创新能力，加速技术和产品迭代升级。

6大核心研究领域

01 智能化软件工程与研发效能

02 AI系统软件技术栈

03 企业开源软件设计与治理

04 软件架构设计与重构

05 系统软件性能工程与优化

06 智能化战略转型与落地

SITS 2026

奇点智能技术大会

Singularity Intelligence Technology Summit

演讲主题 Topic

基于Skill构建规模化Agent任务

张立理

百度前端架构师、百度前端CMC负责人

基于 Skill 构建规模化 Agent 任务

从 Skill 设计原理到大规模任务工程化实践

演讲者：张立理

本次分享的四个主题

01 Skill 是什么

为什么 Agent 需要 Skill？

02 好的 Skill 怎么做

Description 与 Body 各有哪些原则？

03 评测驱动迭代

如何量化 Skill 的质量？

04 长程任务编排

如何让 Agent 完成大规模任务？

PART 01

Skill 是什么

为什么 Agent 需要 Skill？

Agent 不缺能力，缺的是"知道该怎么做"

大模型很聪明，但它有一个根本问题：**没有你的私域知识和专属能力。**

- 你团队的代码规范是什么？
- Code Review 要看哪几个维度？
- 创建 PPTX 应该遵循什么品牌样式？

这些东西不在训练数据里，每次对话都重新教一遍，既低效又不稳定。

> *"即使通过 MCP 给了 Agent 工具调用能力，它依然不知道该按什么流程、什么顺序、什么标准去使用这些工具。"*

一个 Skill 就是一个结构化的文件夹

```
your-skill-name/
  ├── SKILL.md # ← 主入口
  ├── references # ← 其它资源
  ├── scripts
  └── assets
```

命名规则

- 文件夹名用 **kebab-case** :
  - ✓ code-review
  - ✗ Code Review
- 入口文件必须精确命名为 **SKILL.md**

SKILL.md 由两部分组成：Description + Body

```
---
name: my-skill-name
description: 做什么。在用户说"XXX"时使用。
核心能力包括 A、B、C。
---

# My Skill Name

## Instructions

具体的指令内容...
```

两个必填字段：

name

Skill 的唯一标识符，用于系统内部引用和管理。

description

激活判断的核心依据。它常驻于 Agent 的系统提示词中，决定了在什么场景下会加载该 Skill。

渐进式披露

三级结构：

第一级：description（常驻索引）

始终存在于系统提示词，作为激活判断的依据

第二级：SKILL.md 正文（按需加载）

命中后才读取，包含核心指令和工作流程

第三级：references/ 和 scripts/（执行时调用）

只在真正需要时才查阅或执行

为什么要分级？

- **Token 效率**

  不把所有知识一股脑塞进上下文，避免信息过载，节省成本。

- **注意力聚焦**

  模型在长上下文中注意力会衰减，分层保证每阶段只关注最相关的信息，提高执行准确率。

Skill 的工作原理

1. 用户输入

2. Agent 与所有 Skill 的 description 做匹配

3. [工具调用: 读取 SKILL.md]

4. 获得完整指令，开始执行任务

5. [按需读取 references/ 或执行 scripts/]

6. 返回结果

关键事实

- Skill 的激活本身会消耗 1–2 步工具调用。
- Description 写得准不准，直接影响 Token 消耗和响应速度。
- Body 写得不好，自然执行效果不佳，浪费时间和金钱。

PART 02

好的 Skill 怎么做

一个 Skill 能不能用和好不好用，差距巨大。这个差距主要体现在两个地方：Description 决定"什么时候用"，Body 决定"用起来效果如何"。

一个好的 Description 需要同时回答三个问题

01 能做什么

这个 Skill 的核心价值是什么？

示例："分析 Figma 设计稿并生成开发交付文档"

02 核心能力

具体包含哪些能力？

示例："设计规范提取、组件文档生成、标注导出"

03 激活条件

用户说什么话时应该触发？

示例："当用户上传 .fig 文件或要求设计转代码交付时"

缺少激活条件，Agent 不知道什么时候该用；缺少能力描述，Agent 无法判断是否匹配。

Description 的好坏，一眼就能看出来

好的 Description

```
description: >
  通过Figma设计稿生成高还原度的界面，包括素材提取、
  代码脚手架、框架融合、视觉测试。
  当用户提供了.fig稿件，或提到"还原界面"、"实现UI"、
  "F2C"、"Figma2Code"等要求时使用。
```

差的 Description

```
# 太模糊，几乎什么都能匹配
description: 创建Git分支

# 缺少触发条件
description: 创作结构复杂、交互细致的数据可视化报表
```

防止过度触发：加入负向说明

如果 Skill 经常在不相关的场景被加载，可以明确指出"不要用于"什么场景。

```
description: > CSV 文件的高级数据分析。不要用于简单的数据浏览。
```

写 Description 前必须了解的两个事实

Agent 只有在"搞不定"时才找 Skill

如果一个任务 Agent 觉得自己能做（比如写个简单的 Python 脚本），它会直接动手，而不会去查阅 Skill。

应对策略：

Description 必须明确指出这个任务的复杂性或特殊要求，让 Agent 意识到自己"搞不定"。

模型天生偏向"欠触发" (Under-trigger)

为了节省 Token 和避免干扰，模型在不确定是否应该调用工具时，倾向于不调用。

应对策略：

触发条件必须写得极其具体和明确，不能模棱两可，必须给出明确的关键词或场景。

Body 的两种形态

知识文档型

提供规则、标准、检查清单和示例。
适用于单次生成或评估任务。

- Code Review 规范
- PRD 撰写模板
- UI 转代码指南

工作流型

定义步骤、状态流转、校验条件和循环。
适用于多步骤、长程任务。

- 复杂数据分析流
- 大规模代码迁移
- 多阶段内容生成

共同原则：无论哪种形态，核心目的都是向 Agent 明确定义**"什么算做好了"**。

PART 03

评测驱动迭代

写完 Skill 只是第一步，绝不是终点。

我们需要通过系统性的评测来验证其效果，并持续驱动优化。

评测的三个核心原则

核心心法：像对待 Prompt 一样对待 Skill

01 分层评测

不要把触发问题和执行问题混为一谈。Description 评测解决"是否被正确调用"；Body 评测解决"调用后效果好不好"。

02 对照实验

没有对比就没有伤害。必须进行有 Skill vs 无 Skill 的对照测试，证明 Skill 确实带来了增量价值，而不是在帮倒忙。

03 人类参与

机器评测只能作为辅助。最终的质量标准必须由人类专家 (Human in the Loop) 来把关，特别是对于主观性较强的任务。

Description 评测：构建测试集

准备 16–20 条 Query，用于检验 Agent 是否能在正确的时机激活 Skill。

应触发组 (8-10条)

包含直接指令和间接暗示，测试 Agent 的理解能力。

- "帮我 review 一下这段代码"
- "看看这个 PR 有什么问题"

不应触发组 (8-10条)

包含相似但超出范围的请求，测试边界控制能力。

- "帮我写一段快排代码"
  (应直接写，不需 review)
- “我的代码都改了啥"
  (阅读理解)

召回率 (Recall)	精确率 (Precision)
应触发组中，成功触发的比例（防漏触发）	所有触发中，属于应触发组的比例（防误触发）

Description 评测：迭代改进策略

↓ 漏触发 (False Negative)

现象：应该触发 Skill 的 Query，Agent 却没有调用。

策略：推宽边界 — 补充同义词，增加更多具体的场景描述，降低触发门槛。

```
+ "当用户提到'迭代'、'排期'或'Sprint'时使用"
```

↑ 误触发 (False Positive)

现象：不该触发 Skill 的 Query，Agent 却错误地调用了。

策略：收窄范围 — 加入负向约束，明确排除条件，提高触发精确度。

```
+ "注意：不要用于简单的代码解释，仅用于全量 Code Review"
Review"
```

迭代原则：每次只改动一点，重新运行测试集，观察召回率和精确率的变化，直到达到平衡。

Body 评测：五步对照实验

评测 Body 的核心方法是：**有 Skill vs 无 Skill 的对照实验（A/B Test）**。

1

设计用例

准备 5-10 个真实场景的复杂 Query，覆盖核心能力和边界情况。

2

对照实验

使用相同的 Query，分别在"加载 Skill"和"不加载 Skill"状态下执行。

3

定义标准

基于 Body 中的 Critical 和 Quality Checks，制定客观评分维度。

4

评分对比

对两组输出进行盲评打分。如果无显著提升，说明指令无效。

5

分析改进

重点分析 Skill 组中做错的 Case，据此修改 Body。

Body 评测：改进原则

01 保持指令精简

避免冗长的八股文。Agent 不需要你教它常识，只写它容易犯错的地方。指令越长，注意力越容易分散。

02 解释 Why 而不是堆 MUST

与其用十个感叹号和 MUST 强调，不如告诉模型背后的原因。理解了意图，模型能更好地泛化到不同场景。

03 关注重复劳动

如果在评测中发现 Agent 每次都在做相同的机械操作（如格式转换），就应该把它写成自动化脚本，而不是在 Prompt 里反复强调。

04 从反馈中提炼通用规律

不要针对单个失败的 Case 写特例规则（打补丁），要抽象成通用的检查项，防止规则库无限膨胀。

评测闭环

1. 发现问题
2. 修改指令
3. 运行测试
4. 验证提升

持续迭代优化

不要妄想一次写出完美的 Skill，

好 Skill 都是改出来的。

PART 04

长程任务编排

当任务的复杂度超出了单次 Prompt 的能力边界，我们需要将其拆解为多个步骤，并通过长程任务编排来保证最终的交付质量。

什么是长程任务 (Long Running Task)

显著特征

步骤多

通常包含 10 个以上的子步骤，逻辑链路长，依赖关系复杂。

耗时长

执行时间从几十分钟到几个小时不等，跨越多个会话或上下文窗口。

易出错

任何一个中间步骤的失败，都可能导致整个任务前功尽弃。

典型场景示例

- 批量迁移 100 个代码仓库的 CI/CD 配置。
- 爬取 50 个网站的数据并生成结构化报告。
- 自动化排查并修复一个复杂的线上 Bug。

长程任务的三个核心关注点

状态管理

State Management

- 任务进行到哪一步了？
- 产生了哪些中间结果？

错误恢复

Error Recovery

- 某一步失败了怎么办？
- 如何避免从头再来？

边界控制

Boundary Control

- 什么时候该停下来？
- 怎么防止死循环？

长程任务的三个核心困难

01 错误累积

Error Accumulation

现象：

第一步的小偏差，在第十步被放大成灾难。

结果：

最终产出完全不可用，且难以定位最初的错误源头。

02 状态丢失

Context / State Loss

现象：

任务太长，超出了模型的有效上下文窗口；或者模型"忘了"之前的关键决策。

结果：

前后逻辑矛盾，或者重复做已经做过的事。

03 陷入死循环

Infinite Loop

现象：

遇到报错时，模型反复尝试同一种错误的修复方法，无法跳出局部最优解。

结果：

白白烧光 Token，任务永远无法结束。

四个核心原则

01 任务边界清晰

02 错误在最小范围内解决

03 步骤间有校验保障

04 允许局部失败

理念一：任务边界清晰

不要让 **Agent** 自由发挥，要明确告诉它"做完什么就算结束"。

反面教材

"帮我优化一下这个项目。"

后果：Agent 可能会无休止地改下去，修改代码风格、重构架构、甚至引入新的 Bug，永远无法达到"完成"状态。

正面教材

"请检查 src/ 目录下的所有 .ts 文件，将所有的 any 类型替换为具体的类型定义。当你处理完所有文件并运行 npm run build 成功后，任务结束。"

优势：明确了范围（src/ 下的 .ts）、动作（替换 any）、结束条件（ build 成功），Agent 知道何时停止。

理念二：错误在最小范围内解决

核心思想

不要把错误带到下一步。

具体做法

- 在每个子任务内部实现**重试机制**。
- 如果一个文件处理失败，在当前步骤记录错误并重试，而不是等所有文件处理完再统一报错。

核心收益

避免了"因为第 99 个文件失败，导致前 98 个文件的成果被回滚"的悲剧。

理念三：步骤间有校验保障

核心思想：不要盲目相信上一步的结果。在进入下一步之前，必须进行严格的校验。

结构化输出

要求上一步输出 JSON/YAML 等结构化数据，方便下一步进行程序化解析和格式校验。

自动化测试

如果上一步是编写代码，下一步必须是运行单元测试或编译检查，确保代码可用。

交叉验证

如果上一步是提取数据，下一步可以编写一个简短的脚本来检查数据的完整性和一致性。

最终效果：将错误拦截在当前步骤，防止污染后续流程，避免"错误累积"。

理念四：允许局部失败

如果一个长程任务包含 100 个子任务，**不要因为第 99 个失败了，就让整个任务宣告失败。**

1

捕获并记录错误

当某个子任务彻底失败（重试也无效）时，将其记录到错误日志文件中。

error_log.md

2

继续执行

跳过当前失败的子任务，隔离错误范围，继续执行列表中的下一个子任务，保证整体进度推进。

3

最终汇总

在整个任务结束时，向用户报告成功了多少项，失败了多少项，并提供重试失败项的途径。

技巧：File As Progress

核心理念

把进度写在硬盘上，而不是记在脑子里（Context）里。

1

生成进度文件

任务开始前，先生成一个进度追踪文件，列出所有需要执行的子任务。

todo.md / progress.json

2

实时更新状态

每完成一个子任务，立即更新这个文件（打勾或修改状态字段）。

3

中断完美恢复

如果任务中断，下次启动时先读取这个文件，就能完美恢复状态，继续执行未完成的部分。

技巧：子任务 CLI 化与并发调度

核心思想：把长程任务拆解为一个个独立的、可以通过命令行（CLI）调用的子任务。

隔离性

每个子任务在独立的进程中运行，内存和状态隔离。一个子任务崩溃（如 OOM），绝对不会影响主控 Agent 进程。

可测试性

CLI 工具可以脱离 Agent 单独进行单元测试和人工调试，极大降低了长程任务的开发和排错成本。

并发调度

主控 Agent 可以通过并行执行多个 CLI 命令（如使用后台运行或并发工具）来大幅提升整体执行效率。

示例：通过 CLI 独立执行单个仓库的迁移

```
$ python scripts/migrate_repo.py --repo user/repo1 # 成功返回 0，失败返回非 0
```

技巧：多轮重试分层策略

不要每次重试都用一样的 **Prompt**。

第一轮（常规）

正常提供上下文和指令，让 Agent 正常执行任务。

第二轮（带错误信息）

把上一轮的报错信息喂给模型，明确指示它 "Fix the error"。

第三轮（降级/回退）

如果还是失败，尝试使用更简单的方法，或者回退到安全状态。

终极手段（放弃）

如果三次都失败，记录详细日志，跳过当前项，继续下一个任务。

PART 05

Long Running Task Meta Skill

将长程任务的编排经验沉淀为 Meta Skill，让 Agent 自己写长程任务 Skill。

长程任务 Skill 的通用骨架

```
# Long Running Task Skill Template

## 1. Context & Goal
  // 明确任务的最终目标和上下文背景
  You are an expert in [Domain]. Your goal is to [Goal].
## 2. State Management
  // 定义状态保存机制 (File As Progress)
  MUST maintain state in `progress.json`. Read it before starting. Update it after every step.
## 3. Workflow Steps
  // 拆解步骤，定义输入、输出和校验条件
  - Step 1: [Action]. Validation: [Condition].
  - Step 2: [Action]. Validation: [Condition].
## 4. Error Handling
  // 定义重试策略和局部失败处理
  If a step fails, retry up to 3 times. If still failing, log to `error.log` and continue to the next item.
```

Meta Skill：用 Skill 生产 Skill

核心理念：把写 Skill 的经验（如骨架、原则、最佳实践）写成一个 Meta Skill。

"帮我写一个用于批量重构代码的 Skill"

→

Agent 自动调用

Meta Skill

→

自动生成高质量的

SKILL.md

终极意义：实现 Agent 能力的自我繁衍和标准化，极大降低长程任务的开发门槛。

Harness Engineering

什么是 Harness？

在软件工程中，Test Harness（测试线束）是指为了自动化测试而编写的**桩代码、Mock 数据和执行脚本**。

什么是 Harness Engineering？

在 Agent 时代，Harness Engineering 是指：为了让 Agent 能够稳定、高效地完成长程任务，而为其量身定制的**周边脚手架和基础设施**。

核心理念

Agent 不是万能的。
不要试图用 Prompt 解决所有问题，有些问题用代码解决更简单可靠。

把 Agent 擅长的交给 Agent。
让大模型专注于推理、决策、内容理解和代码生成。

把 Agent 不擅长的交给 Harness。
环境准备、状态流转、精确校验、并发调度，这些都应该由脚手架代码来完成。

总结

01. Skill 的核心

Description 解决"何时用"（防漏触发与误触发），Body 解决"怎么用"（知识文档型与工作流型）。

02. 评测驱动迭代

没有 A/B Test 的 Skill 都是盲人摸象。通过构建测试集、分层评测和对照实验，持续优化指令。

03. 长程任务编排

应对错误累积和状态丢失。核心技巧：File As Progress、子任务 CLI 化、多轮重试分层策略。

04. Meta Skill

用魔法打败魔法。将编排经验沉淀为通用骨架，让 Agent 自己写代码、自己编排长程任务。

THANKS

感谢您的聆听与交流